mysql> select schema_name from information_schema.schemata;
TABLES => trouver les noms de tables dans une base de donnée
mysql> select table_name, table_schema from information_schema.tables;
COLUMNS => trouver les noms des colonnes dans une table
mysql> select column_name from information_schema.columns

Lecture de Fichier:

La lecture et surtout l'écriture de fichiers nécessite des privilèges importants pour identifier l'utilisateur actuel on à la fonction user() qu'on peu par exemple appeler avec une union select user() pour lister les droits d'un ou des utilisateurs à la recherche du privilège FILE il faut lire la table information_schema.user_privileges

- Enfin si notre utilisateur possède le privilège FILE la fonction load_file() peut être utilisé pour lire le contenu d'un fichier, cela nécessite aussi évidemment les privilèges nécessaires sur l'OS.

Syntaxe de Base:
SELECT LOAD_FILE("/etc/passwd");
Notons que l'examen du code source de la page par ce moyen pourrait révéler d'autres vulnérabilités intéressantes...

Ecriture de Fichiers:

Venant directement a la création d'une backdoor (webshell) ce privilège est très restreint. En gros oublions ca dans les SGBD modernes
C'est la variable secure_file_priv qui est utilisé pour déterminer d'ou lire/écrire des fichiers.
Pour Obtenir la Valeur de cette Variable:
Select variable_name, variable_value FROM information_schema.global_variables where variable_name="secure_file_priv"

L'instruction SELECT INTO OUTFILE peut être utilisée pour écrire des données issues de requêtes select dans des fichiers
Exemple: SELECT * from users INTO OUTFILE '/tmp/mycredspwn';
On peu aussi select une chaine de caractère arbitraire en écrivant un webshell dans un repertoir précis de l'application vulnerable à l'injection on crée une belle backdoor

Assainissement:

Afin d'éviter ces vulnérabilités il faut "échapper" donc assainir les entrées utilisateurs les SGBD fournissent généralement des fonctions d'assainissement.
comme mysqli_real_escape_string().
ou pour PostgreSQL() pg_escape_string().
Une bonne gestion des pattern et caractères autorisés aide à réduire ce risque
La gestion des privilèges utilisateurs...
Les requêtes paramétrés, (marqueur de position(?) ou le contenu tapé par l'utilisateur sera placé ... / présence de fonction mysqli_stmt_bind_param()
Et aussi les éternels pare-feu web qui font chier plus d'un attaquant.

C'est tout pour ce module le prochain nous apprendras l'utilisation de l'outil SQLMAP, comme toujours, MAITRISER AVANT D'AUTOMATISER